Пермский институт (филиал) федерального государственного бюджетного образовательного учреждения высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Кафедра информационных технологий и программирования

Практическая работа №2

Тема работы: Создание калькулятора с применением библиотеки классов и тестированием.

Работу выполнила:
Александрович Валерия Николаевна
Группа: ИПс-11
Преподаватель: Берестов Дмитрий Борисович

Пермь 2026

Оглавление

Введение. 3
Создание проекта. 4
Создание кода для проекта библиотеки классов. 7
Создание кода для графического интерфейса калькулятора. 8
Создание основного кода. 9
Тестирование. 10
Ручное тестирование калькулятора. 11
Заключение. 13

Введение.

В данной практической работе разрабатывается приложение «Калькулятор» с использованием технологии WPF для создания интерфейса и библиотеки классов для логики вычислений. Калькулятор поддерживает операции сложения, вычитания, умножения, деления и возведения в степень.
Для проверки правильности работы написаны модульные тесты, которые проверяют все методы библиотеки и обработку ошибок, таких как деление на ноль.

Создание проекта.

Я открывала интегрированную среду разработки Visual Studio.

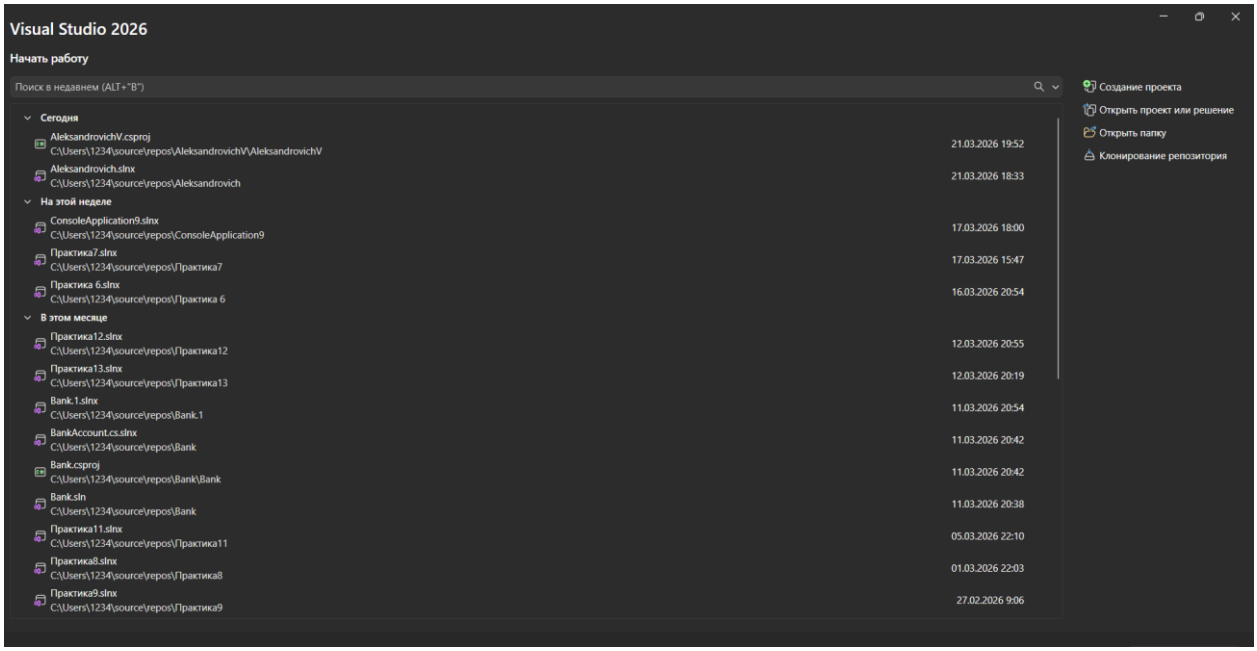

Рисунок 1. Интегрированная среда разработки Visual Studio.

Я создала проект, выбрала «Консольные приложения (Майкрософт)», назвала его своей фамилией Aleksandrovich и первой буквой имени V, получилось AleksandrovichV. Далее выбрала рекомендуемую платформу .NET 10.0 (долгосрочная поддержка).

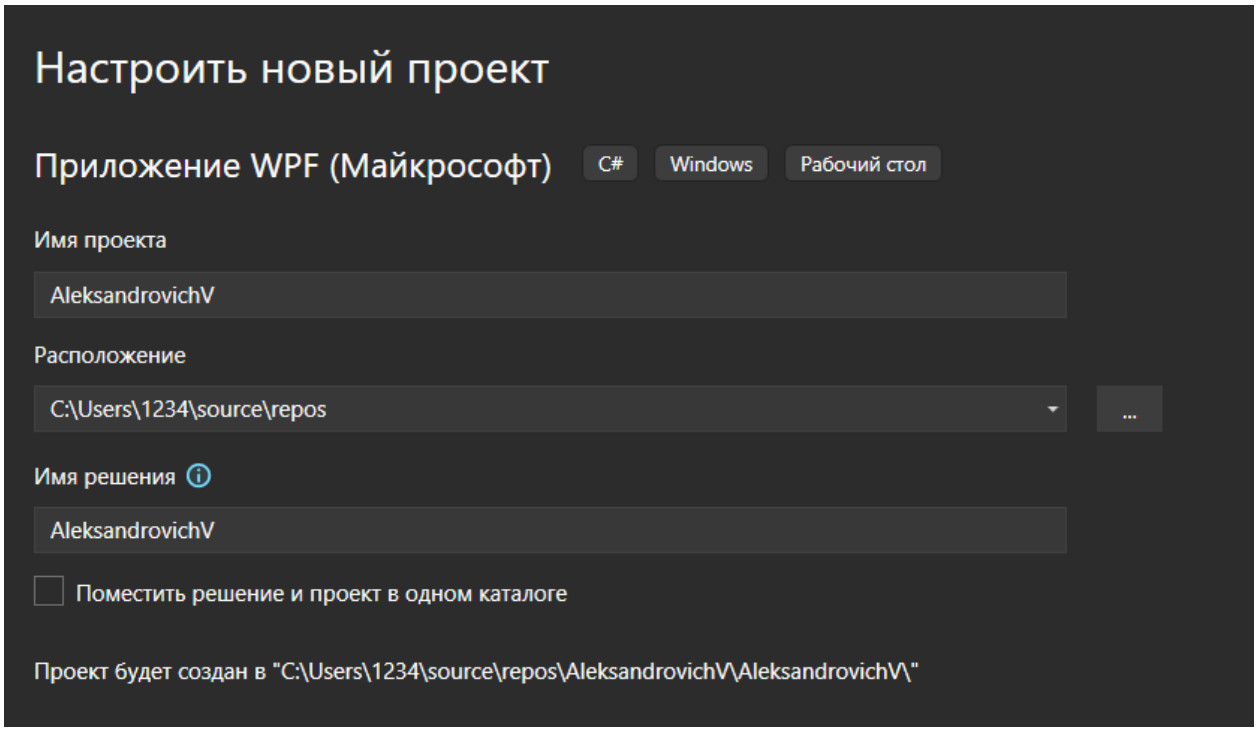

Рисунок 2. Создание проекта «Приложение WPF (Майкрософт)».

Далее в проекте, который был создан нажала на «Вид», затем на «Обозреватель решений», далее нажала правой кнопкой мыши на решение AleksandrovichV2, затем нажала на «Добавить» и «Создать проект», выбрала «Библиотека классов WPF (Майкрософт)», затем назвала проект AleksandrovichLib. Далее выбрала рекомендуемую платформу .NET 10.0 (долгосрочная поддержка).

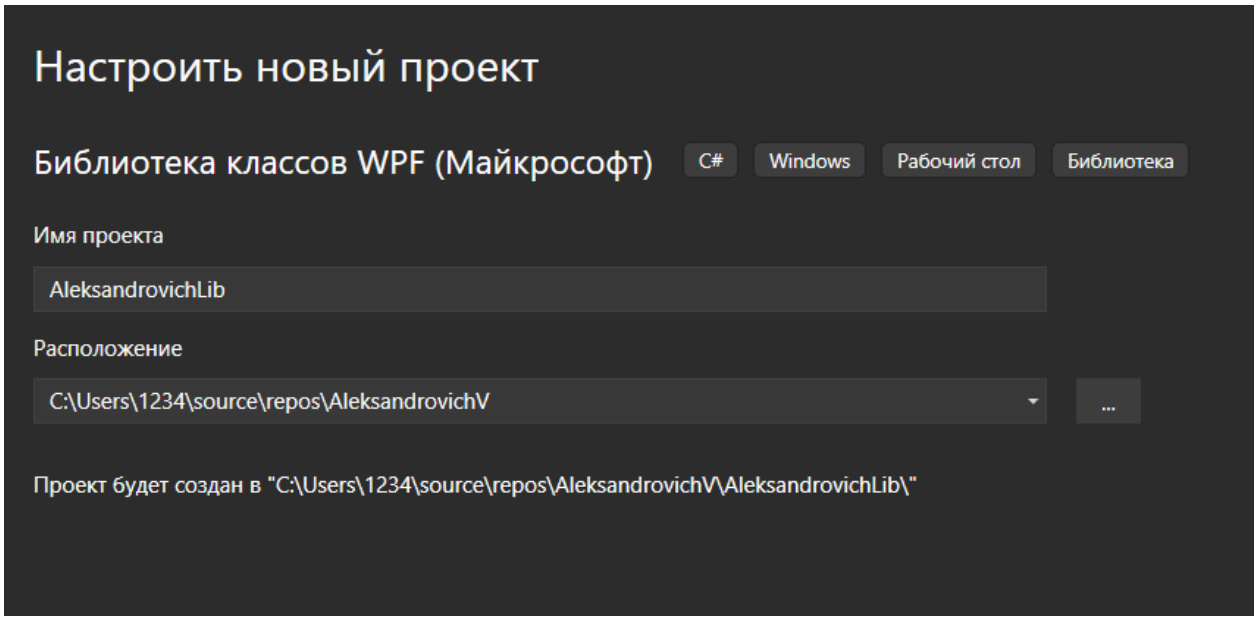

Рисунок 3. Создание проекта «Библиотека классов WPF (Майкрософт)».

Далее нажала правой кнопкой мыши на решение AleksandrovichV2, затем нажала на «Добавить» и «Создать проект», выбрала ««Тестовый проект MSTest (Майкрософт)», затем назвала проект AleksandrovichTests. Далее выбрала рекомендуемую платформу .NET 10.0 (долгосрочная поддержка). Очень важно, чтобы все платформы совпадали, иначе тесты не увидят классы из библиотек, не получится совершить сборку, а также не будет работать тестирование.

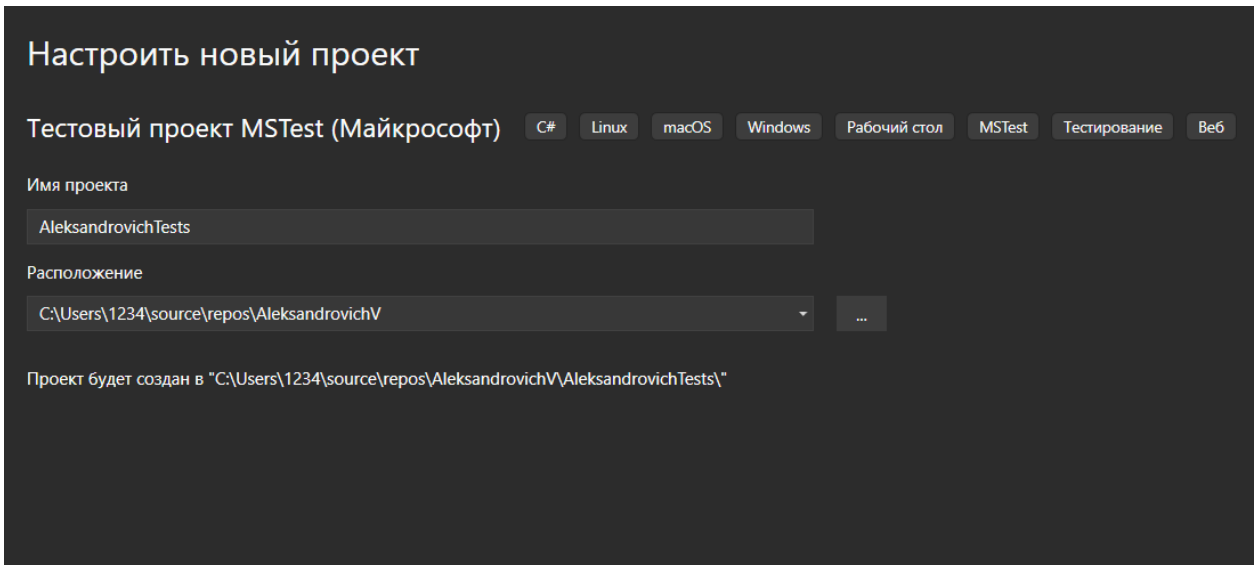

Рисунок 4. Создание проекта «Тестовый проект MSTest (Майкрософт)».

После создания всех необходимых для работы проектов, необходимо добавить ссылку внутри проекта для того, чтобы мой проект тестов смог получить доступ к проекту библиотеки классов. Для этого захожу в «Обозреватель решений», далее в «Зависимости», «Добавить ссылку на проект» и выбираю AleksandrovichLib, так как я подключаю проект библиотека классов к проекту тестов AleksandrovichTests.

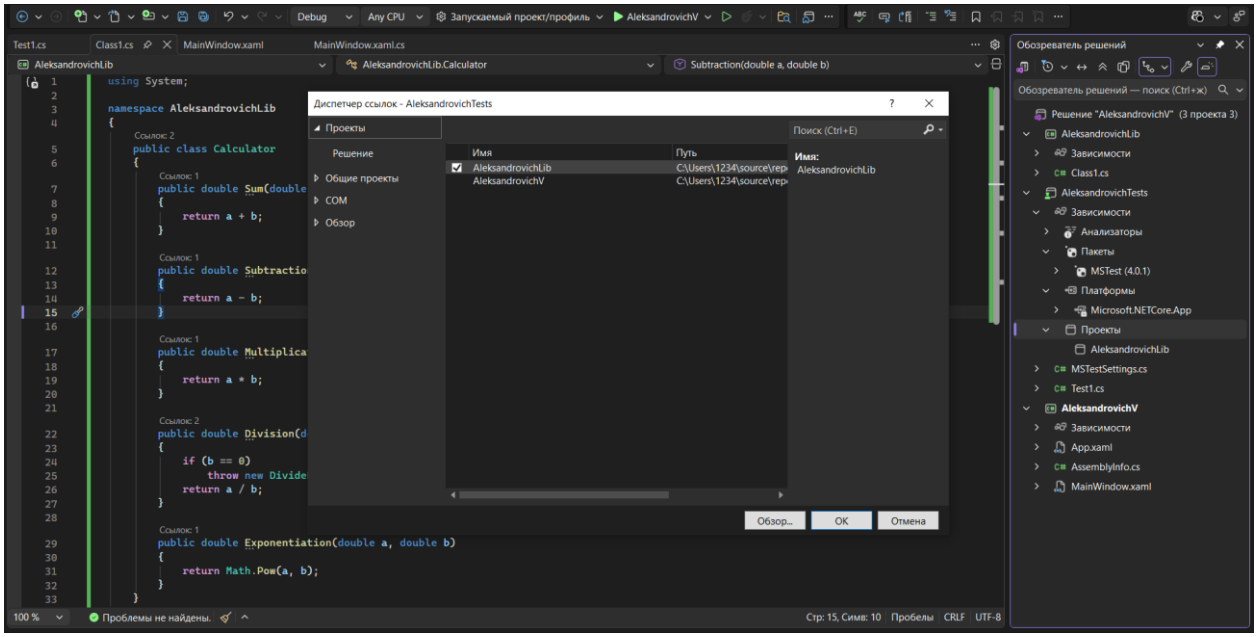

Рисунок 5. Добавление ссылки проекта AleksandrovichLib в проект AleksandrovichTests.

Создание кода для проекта библиотеки классов.

По материалу лекции я ввела пример методов, который был представлен на занятии.
Переименовав класс в Calculator, я приступила к созданию методов арифметических операций.
В библиотеке классов были реализованы четыре метода для выполнения базовых арифметических операций: Sum — сложение; Subtraction — вычитание; Multiplication — умножение; Division — деление (с проверкой делителя на ноль).
Также дополнительно был добавлен метод Exponentiation для возведения в степень.

```
using System;

namespace AleksandrovichLib
{
    Ссылок: 2
    public class Calculator
    {
        Ссылок: 1  1/1 passing
        public double Sum(double a, double b)
        {
            return a + b;
        }

        Ссылок: 1  1/1 passing
        public double Subtraction(double a, double b)
        {
            return a - b;
        }

        Ссылок: 1  1/1 passing
        public double Multiplication(double a, double b)
        {
            return a * b;
        }

        Ссылок: 2  2/2 passing
        public double Division(double a, double b)
        {
            if (b == 0)
                throw new DivideByZeroException("Деление на ноль недопустимо");
            return a / b;
        }

        Ссылок: 1  1/1 passing
        public double Exponentiation(double a, double b)
        {
            return Math.Pow(a, b);
        }
    }
```

Рисунок 6. Код для проекта библиотеки классов.

Создание кода для графического интерфейса калькулятора.

Я создала графический интерфейс самого простого калькулятора.
Ниже представлен рисунок, как выглядел мой калькулятор в конструкторе.

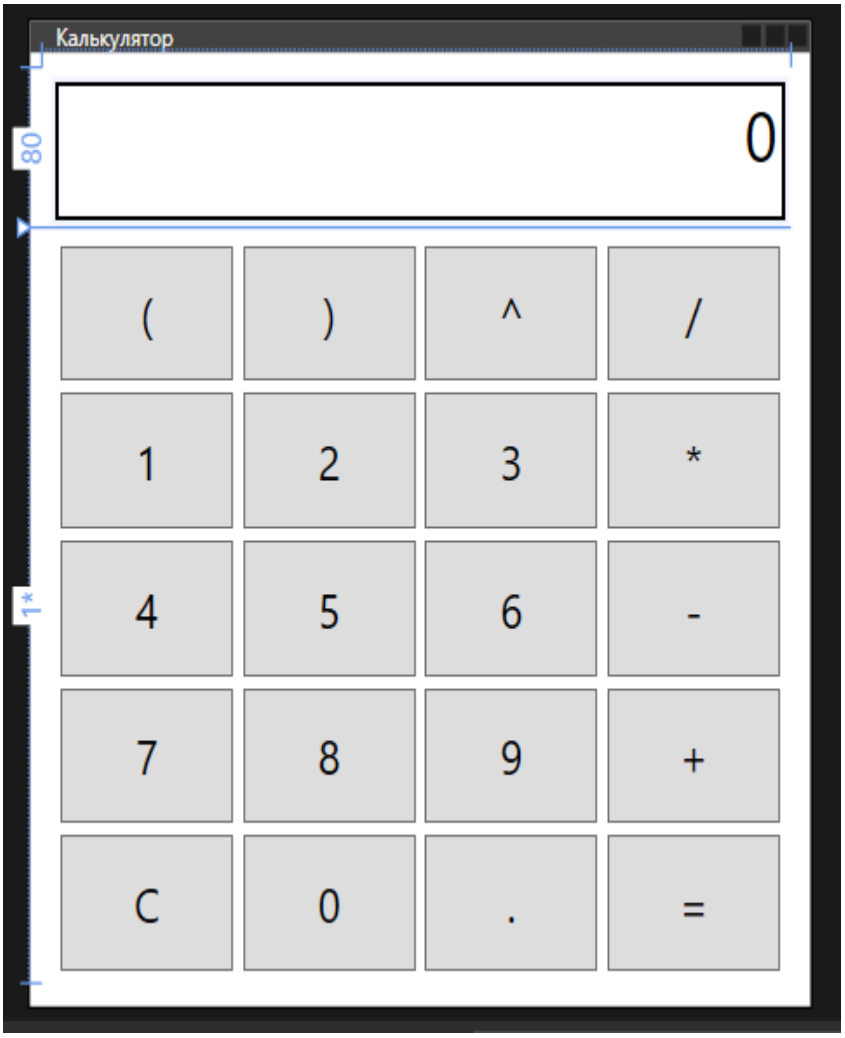

Рисунок 7. Конструктор калькулятора.

Создание основного кода.

Для реализации графического интерфейса и обработки пользовательского ввода было создано WPF-приложение. Основной класс MainWindow содержит обработчики событий для кнопок цифр, операций, скобок, очистки и вычисления результата. Логика вычисления выражения реализована с использованием метода Compute из пространства имён System.Data, который позволяет вычислять строковые математические выражения. Дополнительно были реализованы методы ProcessPower для обработки оператора возведения в степень и ProcessBrackets для рекурсивной обработки скобок. Так же была продумана ошибка деления на ноль, так как в тесте я прописала, что деление на ноль недопустимо, в самом коде я так же прописала при делении на ноль выдаст ошибку.

```
Test1.cs   Class1.cs   MainWindow.xaml   MainWindow.xaml.cs
AleksandrovichV   AleksandrovichV.MainWindow   txtDisplay
7   namespace AleksandrovichV
8   {
        Ссылок: 2
9       public partial class MainWindow : Window
10      {
11          private string expression = "";
12
            Ссылок: 0
13          public MainWindow()
14          {
15              InitializeComponent();
16          }
17
            Ссылок: 10
18          private void Number_Click(object sender, RoutedEventArgs e)
19          {
20              Button btn = (Button)sender;
21              expression += btn.Content.ToString();
22              txtDisplay.Text = expression;
23          }
24
            Ссылок: 5
25          private void Operator_Click(object sender, RoutedEventArgs e)
26          {
27              Button btn = (Button)sender;
28              expression += btn.Content.ToString();
29              txtDisplay.Text = expression;
30          }
31
            Ссылок: 1
32          private void Dot_Click(object sender, RoutedEventArgs e)
33          {
34              expression += ".";
35              txtDisplay.Text = expression;
36          }
37
            Ссылок: 2
38          private void Bracket_Click(object sender, RoutedEventArgs e)
39          {
```

Рисунок 8. Часть основного кода.

Тестирование.

Создаю код для тестирования, затем вижу ошибки, проверяю, что не так и вижу, что платформы у проектов совпадают, а Целевая ОС нет, у проекта тестов она не указана, меняю ОС на Window, так как в библиотеки Window. После редактирования целевой ОС, всё заработало.

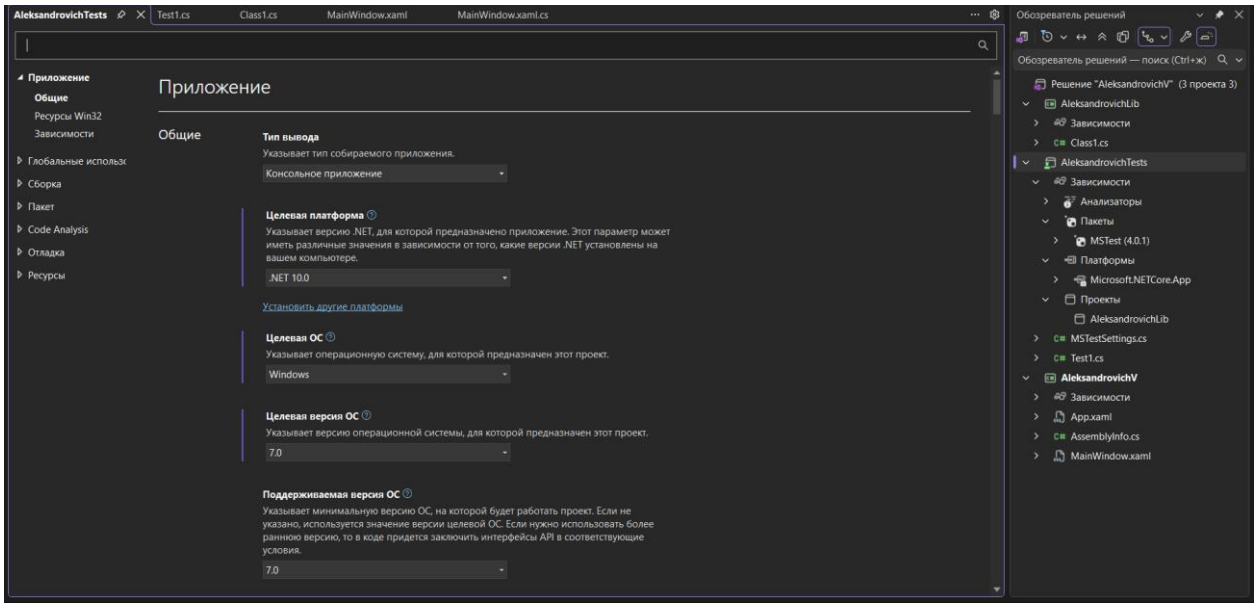

Рисунок 9. Свойства проекта AleksandrovichTests после изменения.

Перед тестирование делаю сборку всех проектов, для этого нажимаю «Сборка», затем «Сборка решения» и так для каждого проекта. Далее нажимаю на «Тест», затем «Запуск всех тестов».
Тестирование выполнено успешно.

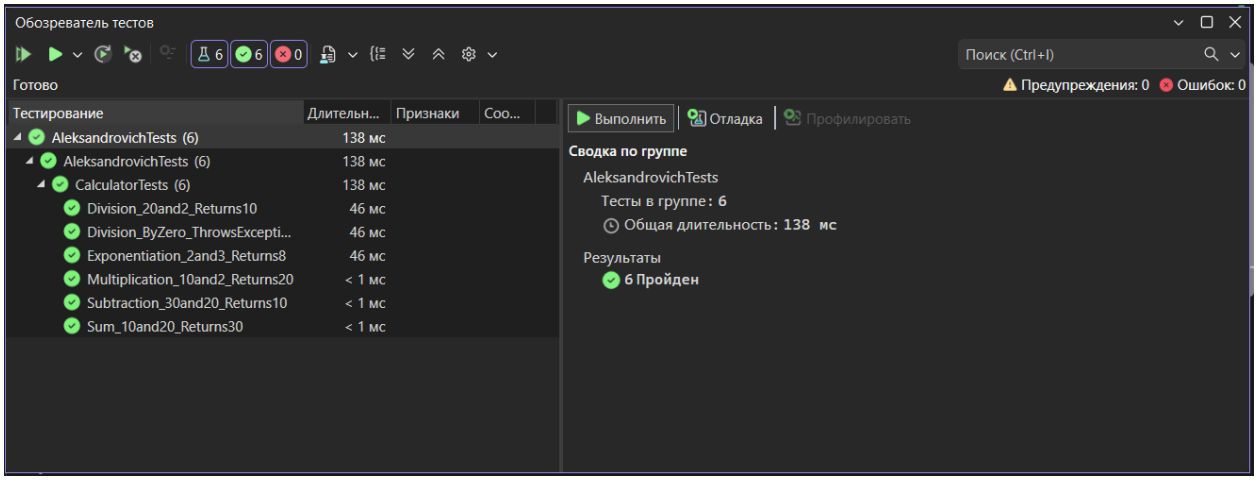

Рисунок 10. Обозреватель тестов.

Редактирование.
Я изменила название проекта с AleksandrovichV на Aleksandrovich.VV.

Ручное тестирование калькулятора.

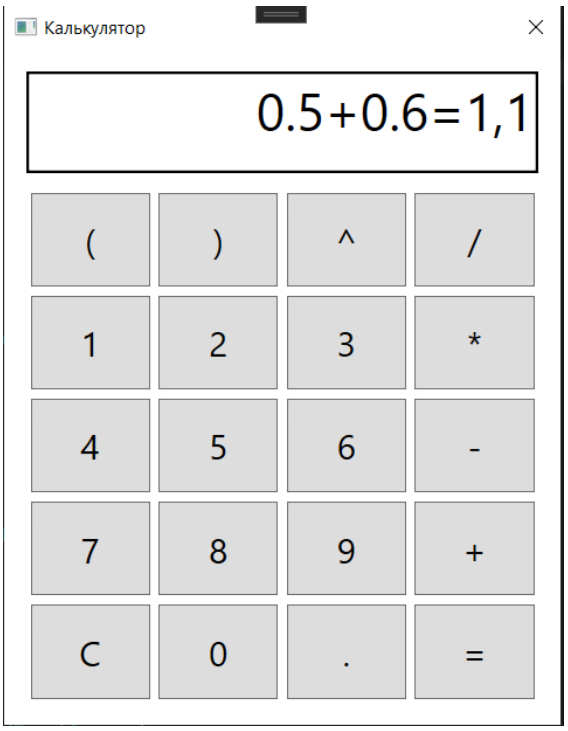

Рисунок 11. Работа калькулятора с плавающей точкой.

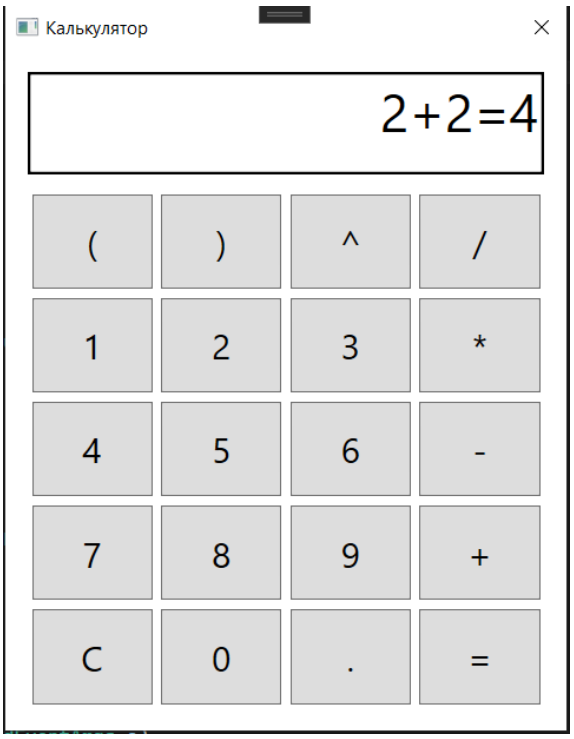

Рисунок 12. Работа калькулятора с простым действием сложения.

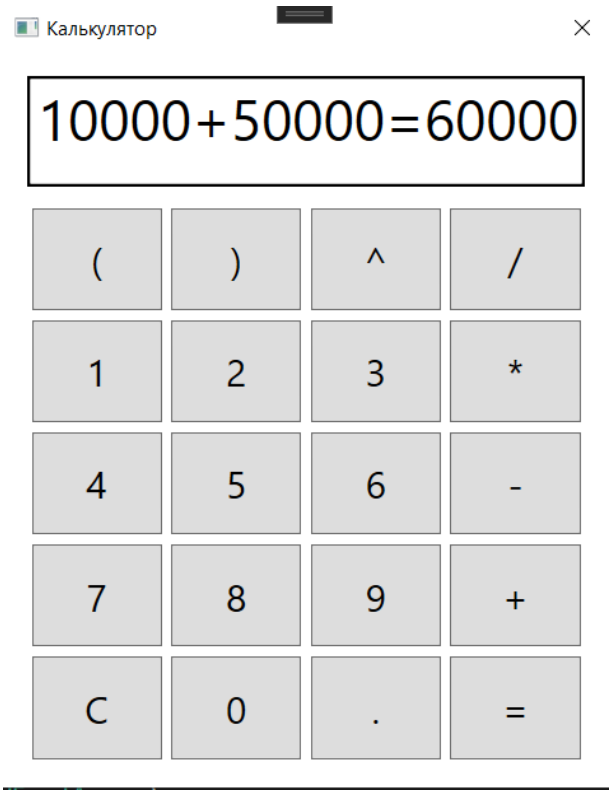

Рисунок 13. Работа калькулятора с большими числами.

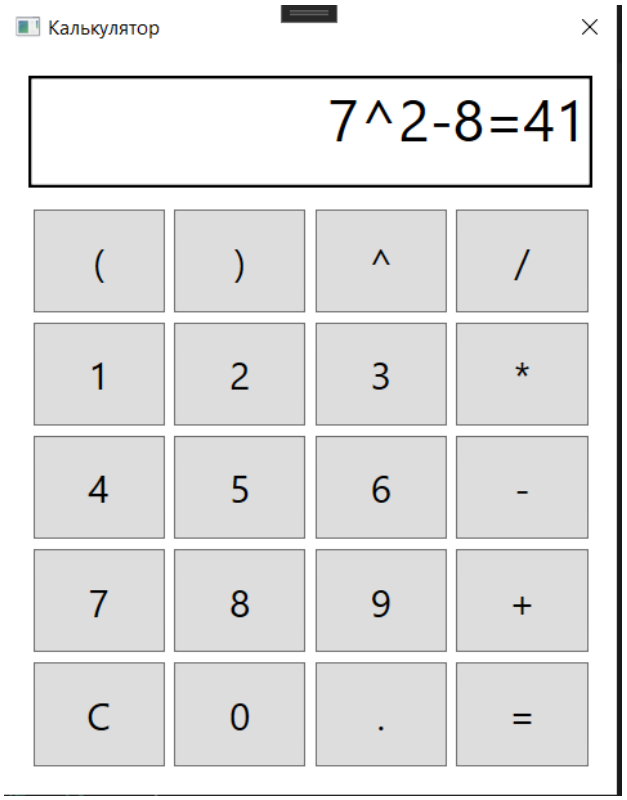

Рисунок 14. Работа калькулятора с двумя действиями: возведение в степень и вычитание.

Заключение.

В ходе работы был создан простой калькулятор, который выполняет сложение, вычитание, умножение, деление и возведение в степень. Интерфейс реализован на WPF, а логика вычислений вынесена в отдельную библиотеку классов.

Для проверки корректности работы были написаны модульные тесты, которые подтверждают правильность всех операций и корректную обработку ошибок, включая деление на ноль.

В результате получилось надёжное и удобное приложение, которое может вычислять простые арифметические операции.

Я научилась подключать библиотеки, осознала важность одинаковых целевых платформ и целевых ос, так как без них не работало ничего, научилась быстро устранять эту ошибку в свойствах проекта.